

SENTINELSHIELD: ADVANCED INTRUSION DETECTION & WEB PROTECTION SYSTEM

PRACTICAL WORK DOCUMENTATION

Abstract

Sentinel Shield is a lightweight Intrusion Detection and Web Protection System developed to simulate the core functionality of a real-world Web Application Firewall (WAF). The project focuses on monitoring incoming HTTP requests, detecting malicious activities, analyzing abnormal traffic behavior, generating alerts, and visualizing attack information through a dashboard interface. The system was designed to provide practical understanding of how cybersecurity monitoring and threat detection mechanisms operate in modern web applications.

Name: **Aman Shaikh**

Email Id: askooli956@gmail.com

Submission Date: **29/06/2026**

Assigned By: ***Cybersecurity Internship Project by “Unified Mentor”***

Index

Sr. No	Title	Page
1	Introduction	2
2	Objective	3
3	Tools & Technologies Used	4
4	System Architecture	5
5	Workflow Explanation	6
6	Attack Detection Rules	7
7	Practical Execution Steps ➤ Execute Safe Request ➤ Execute SQL Injection Attack ➤ Execute XSS Attack ➤ Execute Rate Limit Attack ➤ Execute Header-based Attack ➤ Execute POST Body SQL Injection	8 – 14
8	Detection Results Table	15
9	Dashboard Interpretation	15
10	Log Analysis	15-17
11	Extra Features Added	18
12	Conclusion	18

1. Introduction

About Me: I am **Aman Shaikh** from Pune. I graduated with a **BCA** degree and have expertise in front-end web development using HTML, CSS, JavaScript, Bootstrap, React.js, jQuery libraries, APIs, JSON, and various CDN integrations. I have hands-on experience working with front-end technologies and developing REST APIs using Node.js. I have also started working on basic **penetration testing** for client promotional websites and digital infrastructure (DI) systems.

Purpose of the project: SentinelShield is an Intrusion Detection and Web Protection System developed to simulate the core functionality of a real-world Web Application Firewall (WAF). The project focuses on monitoring incoming HTTP requests, detecting malicious activities, analysing abnormal traffic behaviour, generating alerts, and visualizing attack information through a dashboard UI. The system was designed to provide practical understanding of how cybersecurity monitoring and threat detection mechanisms operate in modern web app.

The application inspects request components such as URLs, headers, query parameters, request bodies, and user agent information to identify common web based attacks including SQL Injection, Cross-Site Scripting (XSS), Local File Inclusion (LFI), Directory Traversal, Command Injection, Suspicious Header Activity, and brute-force or flooding attacks through rate limiting techniques. When suspicious behaviour is detected, the system blocks the request, records detailed logs, generates alerts, and updates the dashboard in real time using Socket.io module in BE.

2. Objective

The main objective of this Project is to understand the working principles of modern Web Application Firewalls and Intrusion Detection Systems through a practical implementation. (As mentioned in unified mentor dashboard)

- To inspect and analyse incoming HTTP requests.
- To detect malicious patterns using rule-based attack signatures.
- To identify common web attacks
- To generate security alerts and detailed logs.
- To visualize attacks using a real-time dashboard.
- To gain practical knowledge of cybersecurity monitoring workflows.

Include:

The project also implements behaviour-based analysis using session tracking and IP monitoring to identify repeated attack attempts and suspicious traffic patterns. Logs are stored with metadata such as timestamps, attack type, request details, IP address, severity level, and attack history, enabling effective security analysis and interpretation. A frontend dashboard developed using React.js and ECharts provides visual summaries including attack distribution, top attacker IPs, recent activities, and real-time notifications.

Add-on:

In addition to signature-based detection, the system includes false-positive analysis, threat insight generation, and sanitized frontend rendering using DOMPurify to improve security and usability. The project demonstrates the practical workflow of attack detection, decision-making, logging, alert generation, and dashboard visualization, providing hands-on experience in cybersecurity operations and web application protection.

3. Tools & Technologies Used

Backend

- Node.js
- Express.js
- Socket.io

Frontend

- React.js
- ECharts
- Tailwind CSS
- DOMPurify

Security Concepts Used

- Signature-based and Behaviour based Detection
- Rate Limiting
- HTTP Request Inspection
- Session Tracking
- Real time Alerting

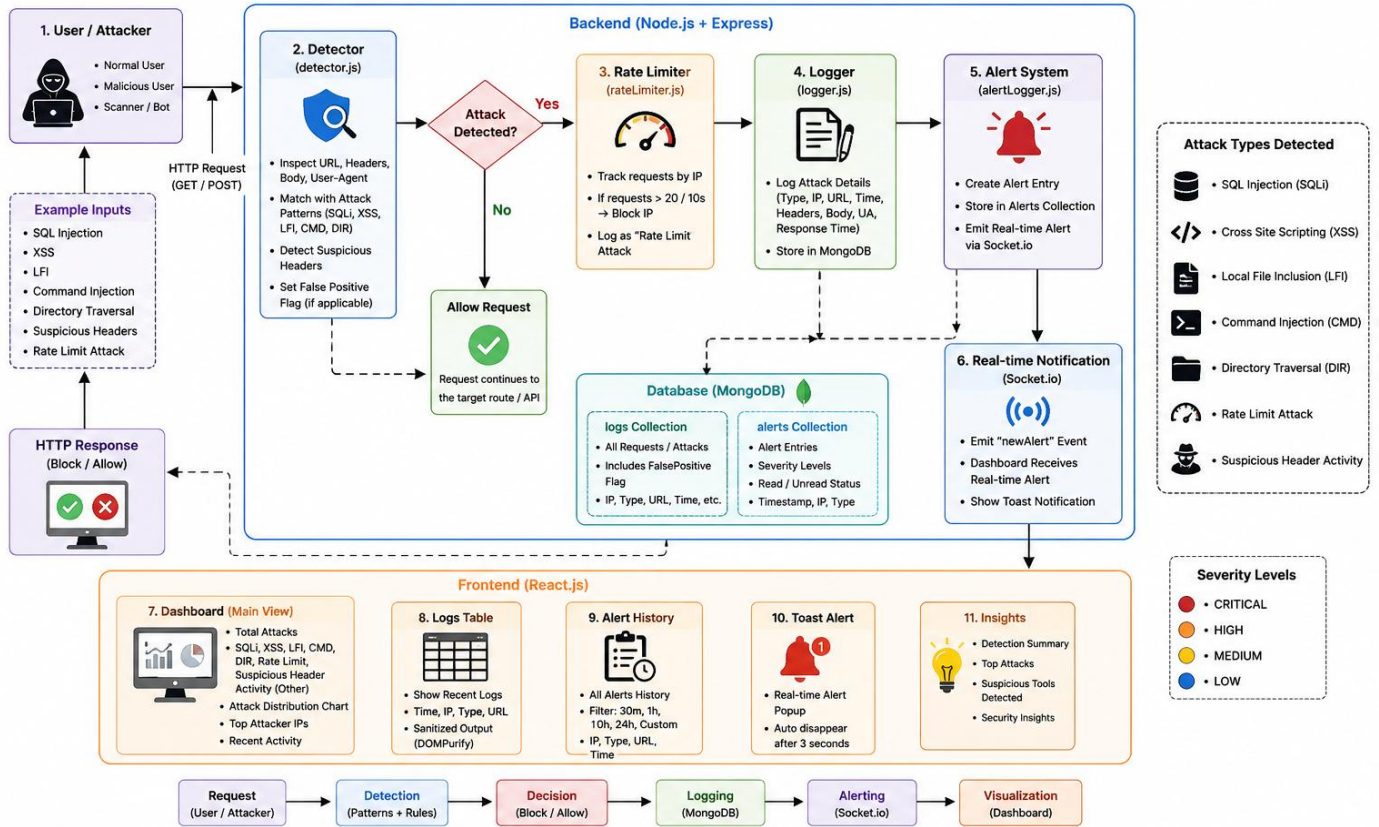
Testing

- curl
- Browser
- Command Prompt / Terminal

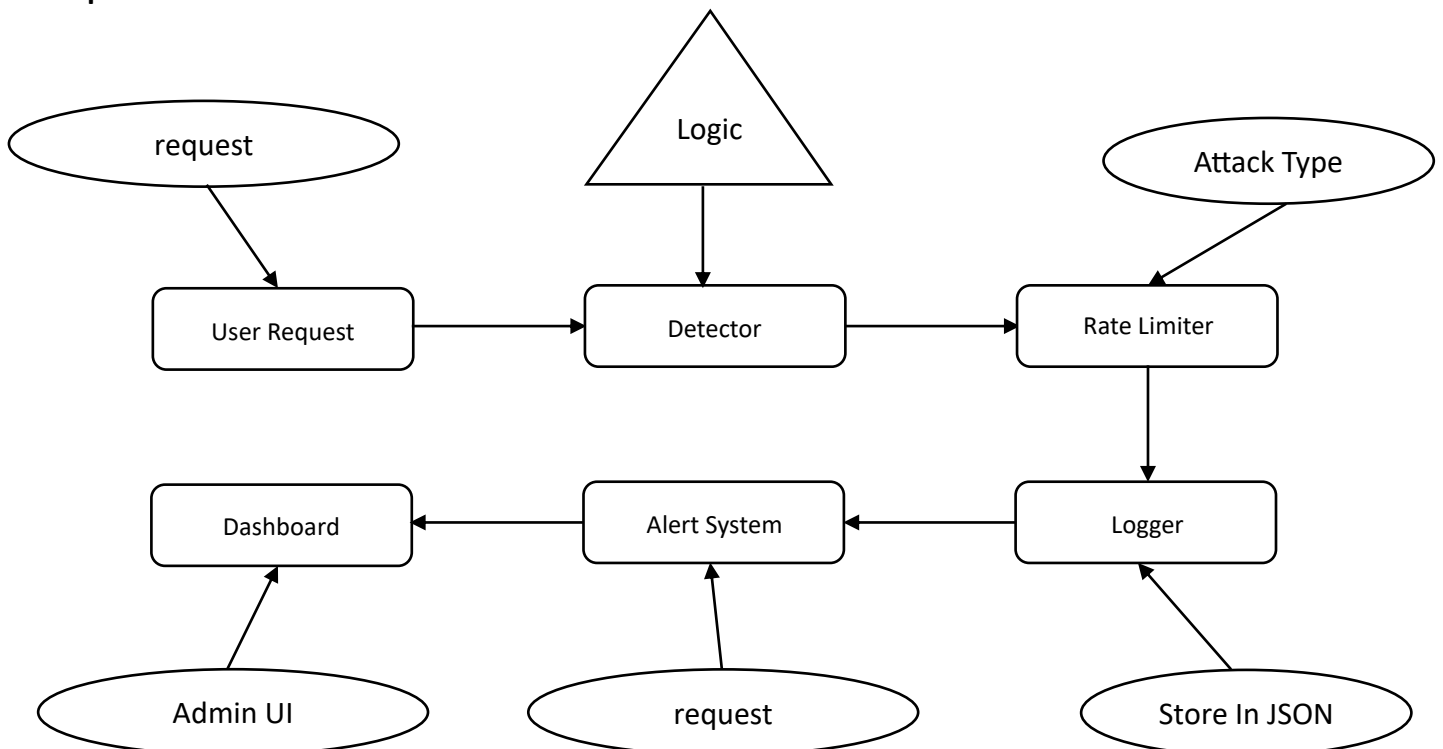
4. System Architecture

SentinelShield – Flow Diagram

(End-to-End Request Flow)



Simple Flow:



5. Workflow Explanation

Explain flow:

1) Request: The server receives incoming HTTP requests from users or attackers.

2) Detector: The detector middleware analyzes URLs, query parameters, headers, request bodies, and user-agent values.

3) Verify Attack Type: The request data is matched against predefined attack signatures.

4) Attack: (Rate Limiting) - The system tracks how many requests an IP sends within a specific time window. If request exceed the allowed limit, the request is marked as abusive. Then the system decides whether the request should be (Allowed/Blocked/Logged & Alerted)

5) Logging and Alerting: If malicious behavior is detected Attack details are logged in Json file and Real time notifications are emitted to dashboard and statistics are updated.

6) Dashboard Visualization: The dashboard visualizes attack distribution, IP activity, alerts, and recent attack history.

6. Attack Detection Rules

When, I done the testing and performed the attack I have used CMD.

The main end-point url:

Local: <http://localhost:3000/test>

Production: <https://sentinelshield-adxf.onrender.com/test>

Attack Type:	Example payload:
SQL Injection	curl http://localhost:3000/login?user=admin%27%20OR%201=1--
XSS	curl <a href="http://localhost:3000/?q=<script>alert(1)</script>">http://localhost:3000/?q=<script>alert(1)</script>
LFI (Local File Inclusion)	curl http://localhost:3000/test?file=/etc/passwd
Directory Traversal	curl http://localhost:3000/test?path=../../etc/passwd
Command Injection	curl http://localhost:3000/test?cmd=test;ls
Rate Limit Attack	for /L %i in (1,1,30) do curl http://localhost:3000/test
Suspicious Header Detection	curl -H "User-Agent: sqlmap" http://localhost:3000/test
POST Body SQL Injection	curl -X POST "http://localhost:3000/test" ^ -H "Content-Type: application/json" ^ -d "{\"username\":\"admin' OR 1=1--\"}"
POST Body XSS	curl -X POST "http://localhost:3000/test" -H "Content-Type: application/json" -d "{\"comment\":\"^<script^>alert(1)^</script^>\"}"

- SQL Injection patterns detected (OR 1=1)
- Rate limiting > brute-force behaviour
- Repeated IP activity detected
- Possible false positives

7. Practical Execution Steps

Step 1: Start Server

Both backend & frontend server is started using “**npm run dev**”

```
AMAAN THE GROOT@LAPTOP-DSM003PE MINGW64 /d/notes/Cyber Security/unifiedmentor intern/sentinelshield (main)
$ cd frontend

AMAAN THE GROOT@LAPTOP-DSM003PE MINGW64 /d/notes/Cyber Security/unifiedmentor intern/sentinelshield/frontend (main)
$ npm run dev

> frontend@0.0.0 dev
> vite

VITE v8.0.8 ready in 1318 ms
→ Local: http://localhost:5173/

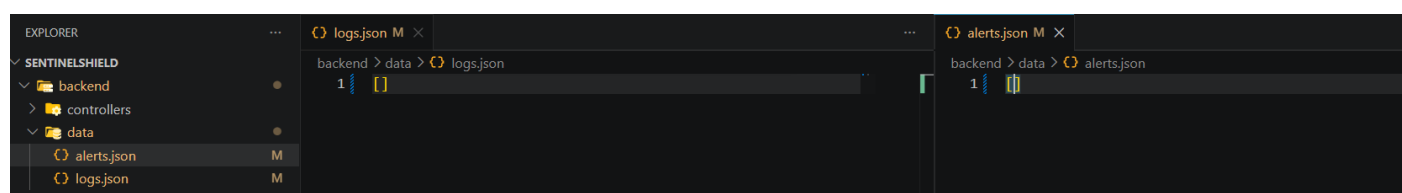
AMAAN THE GROOT@LAPTOP-DSM003PE MINGW64 /d/notes/Cyber Security/unifiedmentor intern/sentinelshield/backend (main)
$ npm run dev

> sentinelshield@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
Server running on port 3000
```

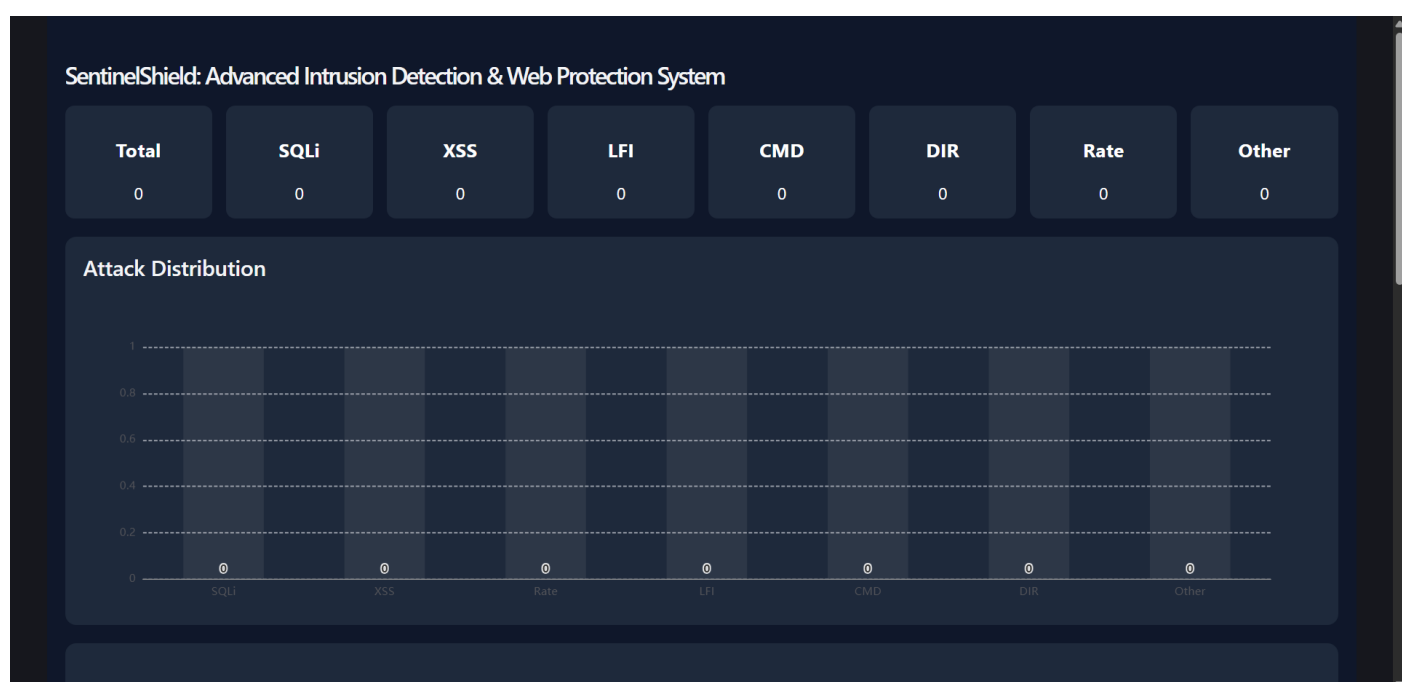
IMP: As mentioned in the project description “Students manually inspect log entries and categorize alerts.” Using json file to store Data

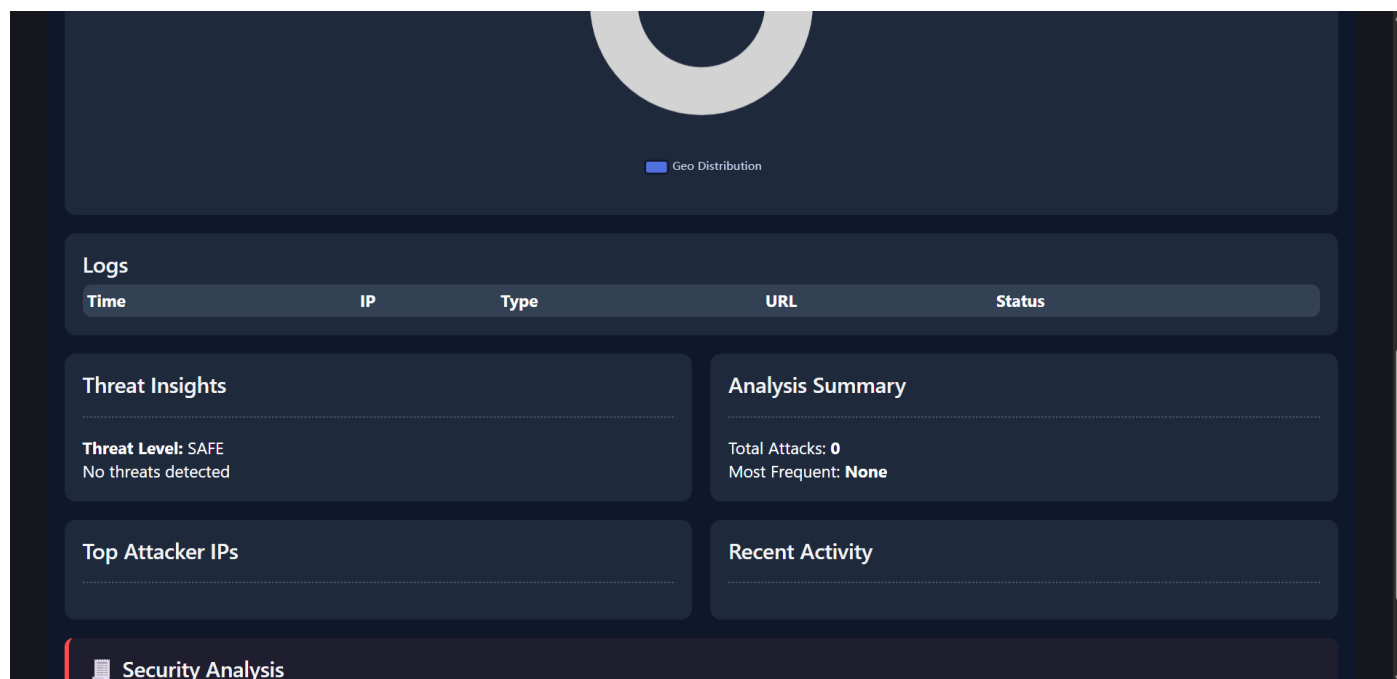
Both the file is empty for now logs.json: [] and alerts.json: []



Step 2: Open Dashboard

Frontend app is running

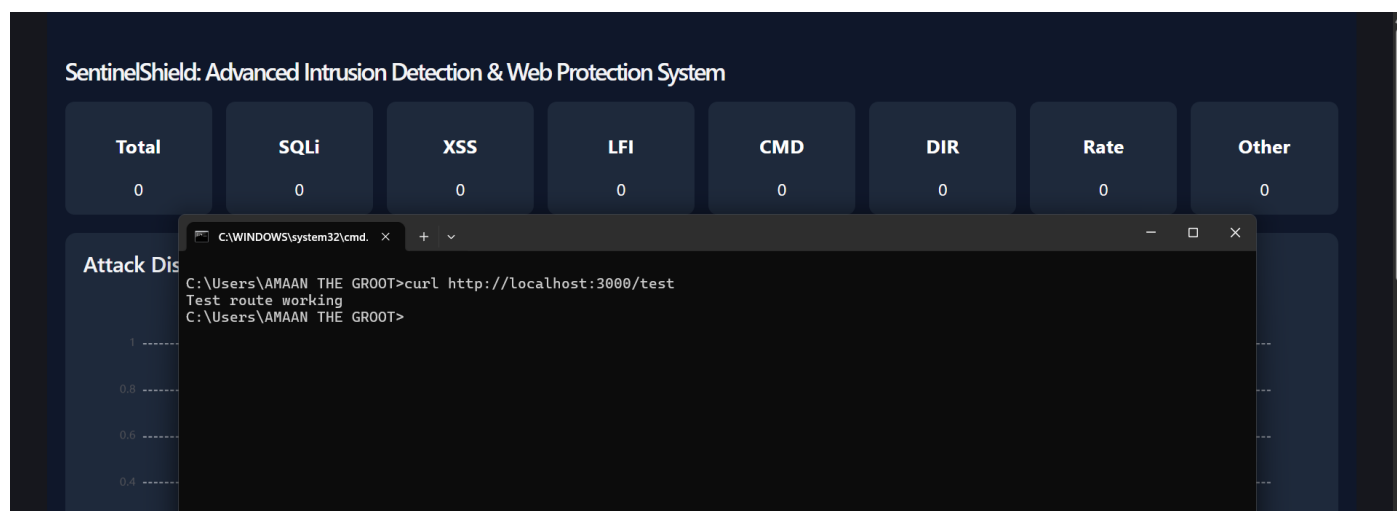




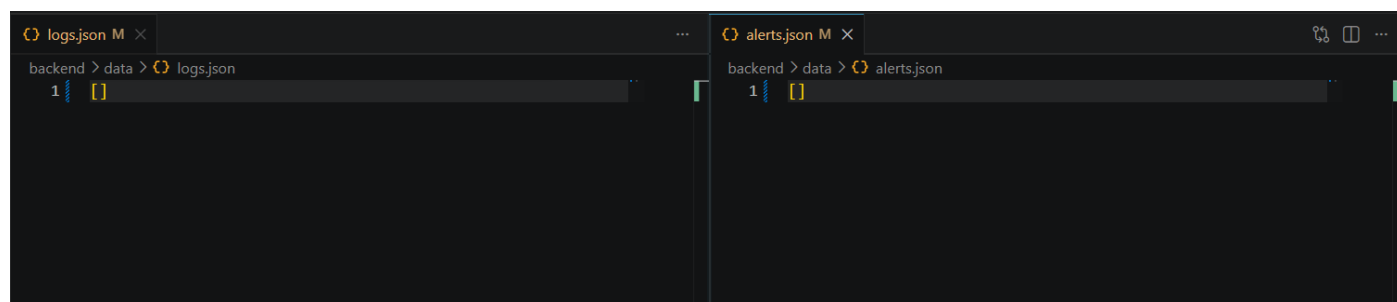
Step 3: Execute Safe Request

Command: curl <http://localhost:3000/test>

Expected Result: Request allowed & No alerts generated



No alerts

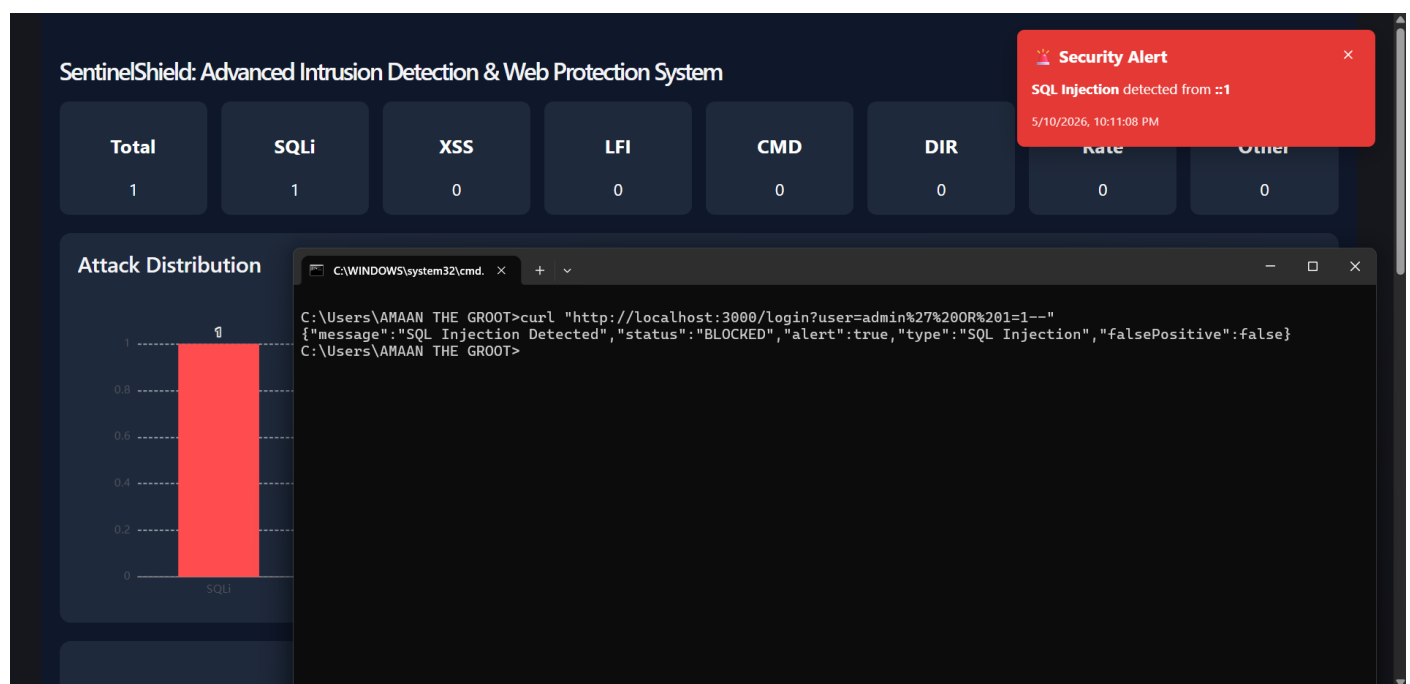


Step 4: Execute SQL Injection Attack

Command: `curl "http://localhost:3000/login?user=admin%27%20OR%201=1--"`

Expected Result: Request blocked SQL > Injection detected > Log generated > Alert generated

Request blocked SQL > Injection detected > Alert generated.



Log generated

```
logs.json M X
backend > data > logs.json > ...
1 [
2   {
3     "requestId": "req_1778431268162",
4     "time": "2026-05-10T16:41:08.166Z",
5     "ip": "::1",
6     "ipStatus": "Normal",
7     "geo": {
8       "country": "Local",
9       "city": "Local",
10      "isp": "Local Machine"
11    },
12    "type": "SQL Injection",
13    "pattern": "SQL Injection via request",
14    "payload": "{\"user\": \"admin' OR 1=1--\"}",
15    "url": "/login?user=admin%27%20OR%201=1--",
  }
]

alerts.json M X
backend > data > alerts.json > ...
1 [
2   {
3     "requestId": "req_1778431268670",
4     "time": "2026-05-10T16:41:08.670Z",
5     "ip": "::1",
6     "type": "SQL Injection",
7     "url": "/login?user=admin%27%20OR%201=1--",
8     "severity": "CRITICAL",
9     "count": 1,
10    "lastTriggered": "2026-05-10T16:41:08.166Z",
11    "method": "GET",
12    "userAgent": "curl/8.18.0",
13    "markAsRead": "No",
14    "sendToAdmin": "Yes"
15  }
]
```

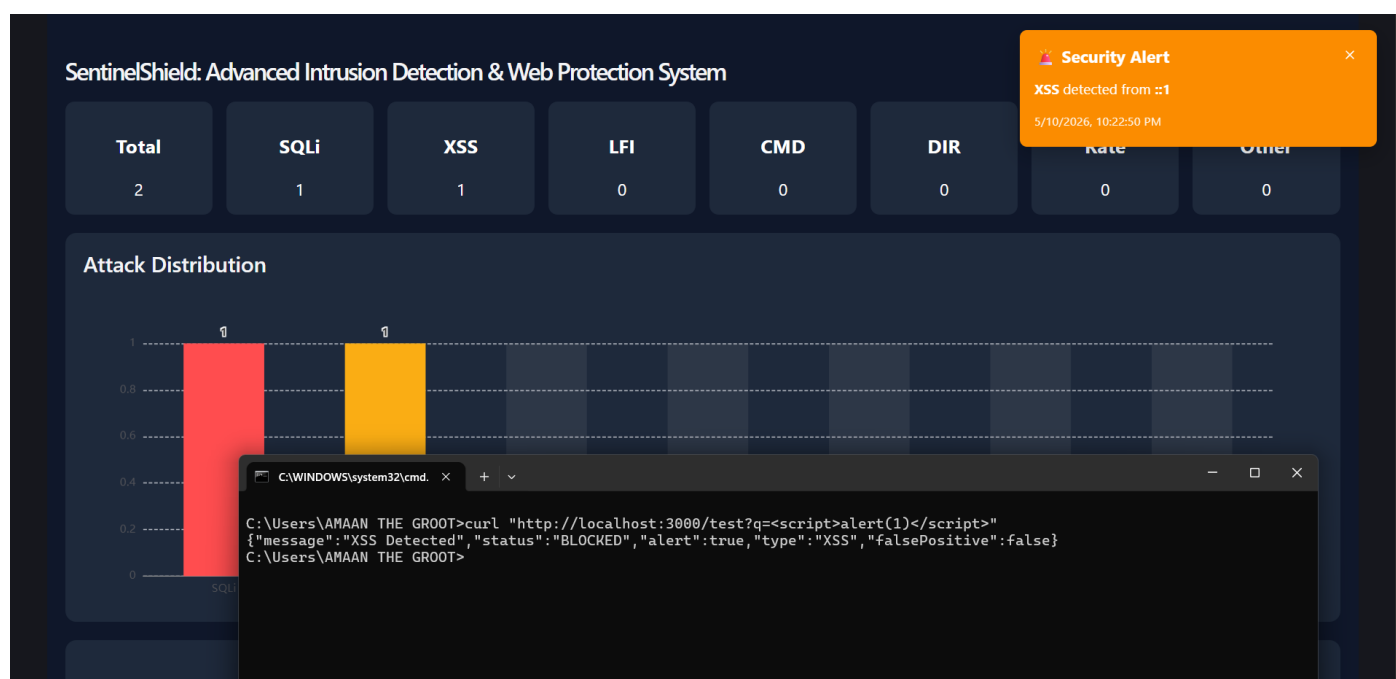
Backend logs

```
edmentor intern/sentinelshield/backend (main)
$ npm run dev
Client Socket connected
Client Socket disconnected
Client Socket connected
Client Socket disconnected
Client Socket connected
ALERT: SQL Injection detected from ::1
Client Socket disconnected
Client Socket connected
ALERT: SQL Injection detected from ::1
```

Step 5: Execute XSS Attack

Command: `curl "http://localhost:3000/test?q=<script>alert(1)</script>"`

Expected Result: Request blocked > XSS detected



Log generated

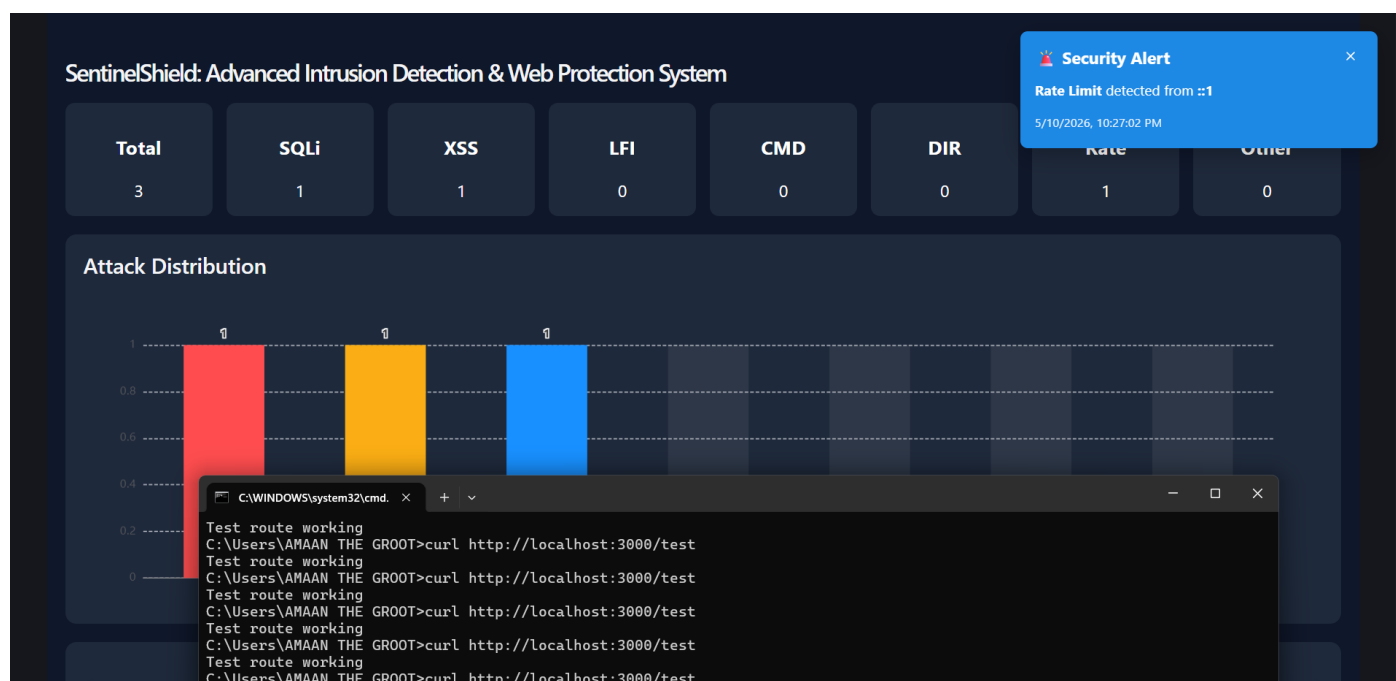
```
logs.json M X alerts.json M X
backend > data > logs.json > ... backend > data > alerts.json > ...
43 { 2 {
52 }, 15 {
53 "type": "xss", 16 {
54 "pattern": "XSS via request", 17 "requestId": "req_1778431970587",
55 "payload": "{ \"q\": \"<script>alert(1)</script>\" }", 18 "time": "2026-05-10T16:52:50.587Z",
56 "url": "/test?q=<script>alert(1)</script>", 19 "ip": "::1",
57 "sourceType": "Script", 20 "type": "XSS",
58 "fingerprint": "::1_curl_xss", 21 "url": "/test?q=<script>alert(1)</script>",
59 "severity": "HIGH", 22 "severity": "HIGH",
60 "riskScore": 7. 23 "count": 1,
```

```
Client Socket disconnected
Client Socket connected
ALERT: SQL Injection detected from ::1
ALERT: XSS detected from ::1
```

Step 6: Execute Rate Limit Attack

Command: `for /L %i in (1,1,30) do curl http://localhost:3000/test`

Expected Result: Request blocked SQL > Injection detected > Log generated > Alert generated



logs.json

```
backend > data > logs.json > {} 2
84 {
94 },
95 "type": "Rate Limit",
96 "severity": "Low",
97 "url": "/test",
98 "payload": "N/A",
99 "sourceType": "Script",
100 "fingerprint": "::1_curl_RateLimit",
101 "riskScore": 3,
```

alerts.json

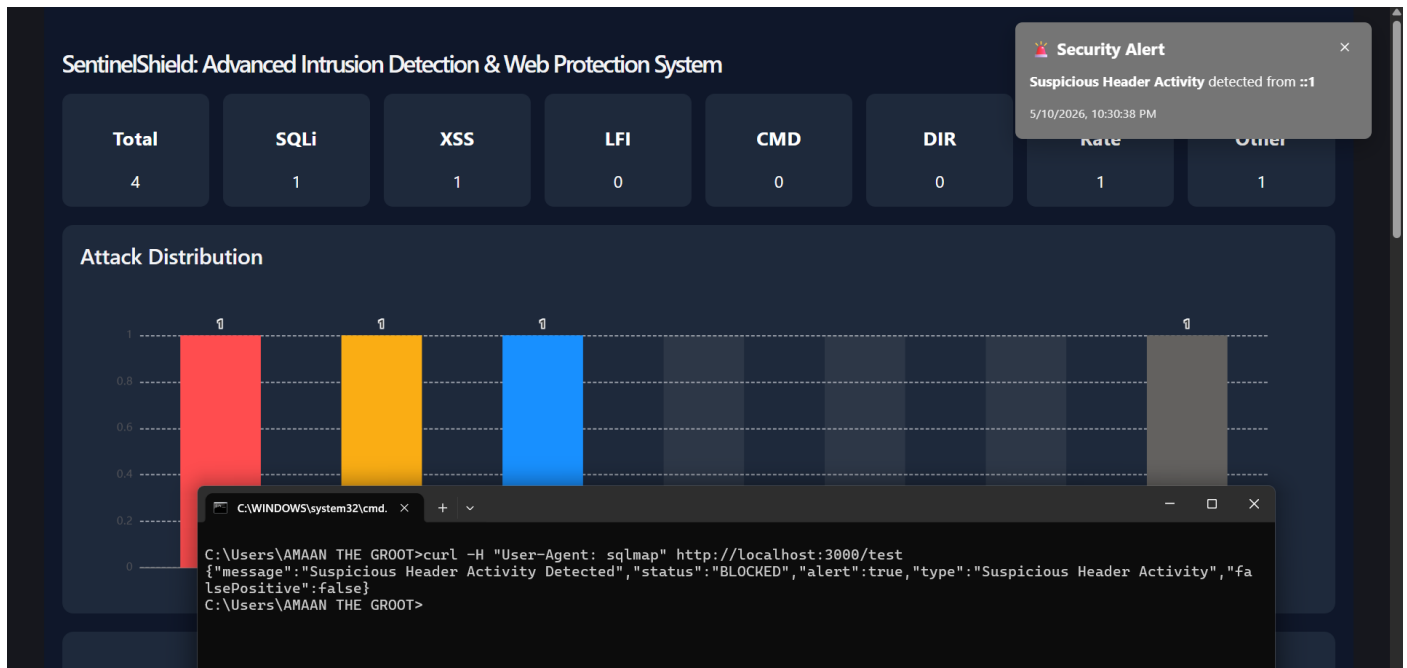
```
backend > data > alerts.json > {} 2
30 {
31 "requestId": "req_1778432222244",
32 "time": "2026-05-10T16:57:02.244Z",
33 "ip": "::1",
34 "type": "Rate Limit",
35 "url": "/test",
36 "severity": "Low",
37 "count": 1,
38 "lastTriggered": "2026-05-10T16:57:01.731Z",
39 "method": "GET"
```

```
Client Socket disconnected
Client Socket connected
🚨 ALERT: Rate Limit detected from ::1
```

Step 7: Execute Header-based Attack

Command: `curl -H "User-Agent: sqlmap" http://localhost:3000/test`

Expected Result: Suspicious Header Activity detected > Injection detected > Log generated > Alert generated



The screenshot shows two JSON files side-by-side. The left file, 'logs.json', contains a log entry for 'Suspicious Header Activity' with details like 'city': 'Local', 'isp': 'Local Machine', 'pattern': 'Suspicious Header Activity via request', 'payload': 'N/A', 'url': '/test', and 'sourceType': 'Attack Tool'. The right file, 'alerts.json', contains an alert entry with 'sendToAdmin': 'Yes', 'requestId': 'req_1778432438279', 'time': '2026-05-10T17:00:38.279Z', 'ip': '::1', 'type': 'Suspicious Header Activity', 'url': '/test', 'severity': 'UNKNOWN', and 'count': 1.

```
backend > data > logs.json > {} 3
287 {
292   "geo": {
294     "city": "Local",
295     "isp": "Local Machine"
296   },
297   "type": "Suspicious Header Activity",
298   "pattern": "Suspicious Header Activity via request",
299   "payload": "N/A",
300   "url": "/test",
301   "sourceType": "Attack Tool",
302   "falsePositive": false
303 }

backend > data > alerts.json > {} 3
30 {
42   "sendToAdmin": "Yes"
43 },
44 },
45   "requestId": "req_1778432438279",
46   "time": "2026-05-10T17:00:38.279Z",
47   "ip": "::1",
48   "type": "Suspicious Header Activity",
49   "url": "/test",
50   "severity": "UNKNOWN",
51   "count": 1
52 }
```

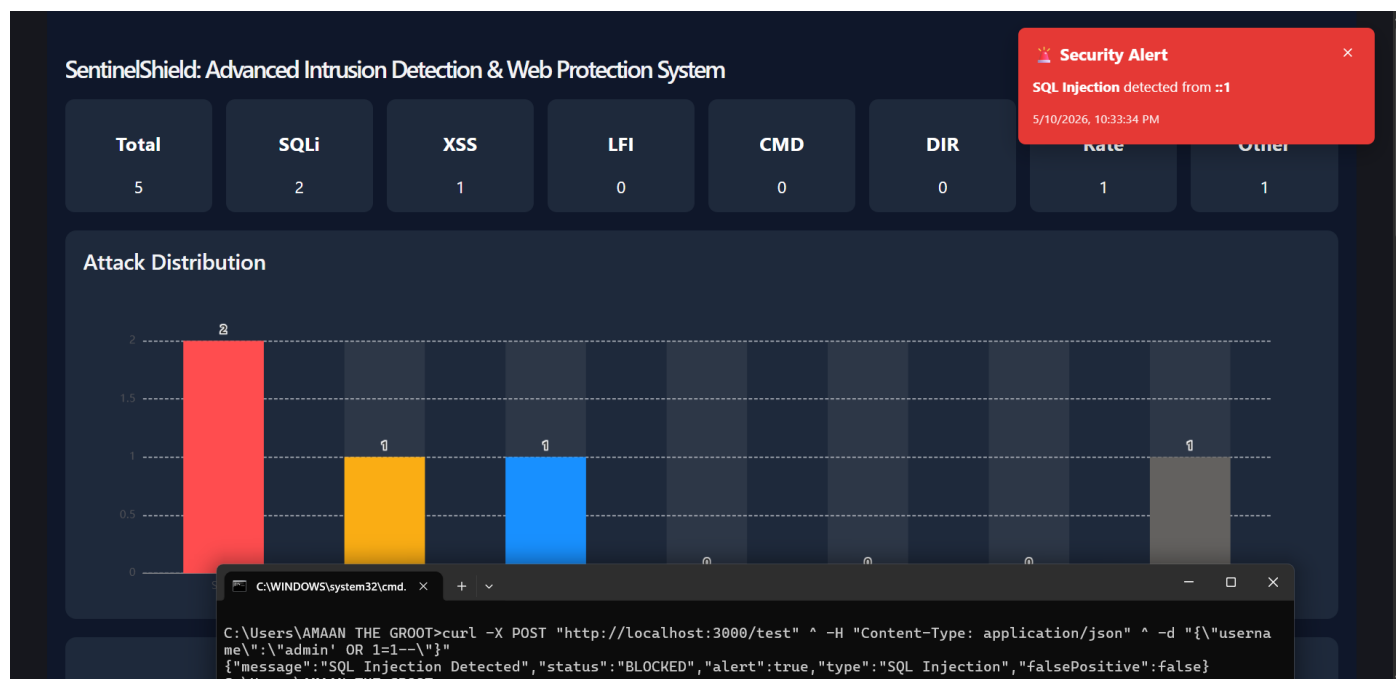
The screenshot shows a terminal window with the message: 'ALERT: Suspicious Header Activity detected from ::1'.

```
ALERT: Suspicious Header Activity detected from ::1
```

Step 8: Execute POST Body SQL Injection

Command: `curl -X POST "http://localhost:3000/test" ^ -H "Content-Type: application/json" ^ -d '{"username":"'admin' OR 1=1--\"}'`

Expected Result: SQL Injection detected > Request blocked > Log generated > Alert generated



The screenshot shows two log files, `logs.json` and `alerts.json`, with the detected SQL Injection attack details.

logs.json:

```
335 },
336 "type": "SQL Injection",
337 "pattern": "SQL Injection via request",
338 "payload": "{\"username\":\"'admin' OR 1=1--\"}",
339 "url": "/test",
340 "sourceType": "Script",
341 "fingerprint": "::1_curl_SQLInjection",
342 "severity": "CRITICAL",
343 "riskScore": 9,
```

alerts.json:

```
57 },
58
59 "requestId": "req_1778432614539",
60 "time": "2026-05-10T17:03:34.539Z",
61 "ip": "::1",
62 "type": "SQL Injection",
63 "url": "/test",
64 "severity": "CRITICAL",
65 "count": 1,
```

The screenshot shows the terminal output of the alert:

```
ALERT: SQL Injection detected from ::1
```

Below the alert, the terminal status bar shows: `unknown (23 hours ago) Ln 71, Col 4 Spaces: 2 UTF-8 LF { } JSON (Go Live)`

8. Detection Results Table

Attack Type	Detected	Blocked	Logged	Alert Generated
SQL Injection	Yes	Yes	Yes	Yes
XSS	Yes	Yes	Yes	Yes
LFI	Yes	Yes	Yes	Yes
Command Injection	Yes	Yes	Yes	Yes
Directory Traversal	Yes	Yes	Yes	Yes
Rate Limit Attack	Yes	Yes	Yes	Yes
Suspicious Header Activity	Yes	Yes	Yes	Yes
POST Body SQLi	Yes	Yes	Yes	Yes
POST Body XSS	Yes	Yes	Yes	Yes

9. Dashboard Interpretation

The frontend dashboard provides real-time visualization of attack activities.

Dashboard Features

Attack Distribution Chart: Displays the number of attacks categorized by attack type.

Top Attacker IPs: Displays IP addresses responsible for repeated attacks.

Real-time Toast Alerts: Displays instant notifications whenever malicious activity is detected.

Recent Activity Logs: Shows rece

10. Log Analysis

The SentinelShield system maintains detailed security logs for every detected malicious request. These logs help analyze attacker behavior, identify suspicious activity patterns, and understand how the system responds to threats.

Information Stored in Logs

Each log entry contains:

- Timestamp of the attack
- IP address of the requester
- Attack type
- Request URL
- Request payload
- Request method

- User-Agent information
- Severity level
- Risk score
- Session history
- Detection status

Example log details:

```
{
  "requestId": "req_1778431268162",
  "time": "2026-05-10T16:41:08.162Z",
  "ip": "::1",
  "ipStatus": "Normal",
  "geo": {
    "country": "Local",
    "city": "Local",
    "isp": "Local Machine"
  },
  "type": "SQL Injection",
  "pattern": "SQL Injection via request",
  "payload": "{\"user\":\"admin' OR 1=1--\"}",
  "url": "/login?user=admin%27%20OR%201=1--",
  "sourceType": "Script",
  "fingerprint": "::1_curl_SQLInjection",
  "severity": "CRITICAL",
  "riskScore": 9,
  "detectionTimeMs": 1,
  "responseTimeMs": 1,
  "count": 1,
  "lastTriggered": "2026-05-10T16:41:08.162Z",
```

```
"attackStage": "Detected → Blocked",
"status": "Active",
"falsePositive": false,
"request": {
  "method": "GET",
  "userAgent": "curl/8.18.0",
  "headers": {
    "host": "localhost:3000",
    "user-agent": "curl/8.18.0",
    "accept": "*/*"
  },
  "query": {
    "user": "admin' OR 1=1--"
  },
  "body": null
},
"method": "GET",
"userAgent": "curl/8.18.0"
}
```

The system performs session-based behavior monitoring to identify repeated attack attempts from the same IP address.

Example Workflow

- First attack session > IP marked as “Normal”
- Repeated attack sessions > IP marked as “Suspicious”
- Continuous abusive behavior > Higher risk score assigned

11. Extra Features Added

- **Real-time Alerts using Socket.io**
- **Session-based Attack Detection** (Instead of treating every repeated request as a separate attack, the system groups repeated malicious requests into attack sessions. This helps identify behavioral attack patterns more accurately.)
- **Threat Insights Panel:** (A threat insights section was added to the dashboard to provide analytical summaries)
- **Suspicious Header Detection:** (The system detects automated scanning tools such as sqlmap, nmap by analyzing the User-Agent header.)
- **POST Body Attack Detection:** (The detector analyzes request body payloads for malicious content such as SQL Injection and XSS attacks submitted using POST requests.)
- **Session History Tracking:** (Repeated attacks from the same IP are stored in history logs to support behavioral analysis and attack investigation.)
- **Severity-based Alerting:** (Alerts are categorized into different severity levels: CRITICAL/HIGH/MEDIUM/ LOW This helps prioritize security incidents.)
- **DOMPurify Sanitization:** (The frontend uses DOMPurify to sanitize dynamic content before rendering logs and alerts, reducing the risk of frontend-based XSS attacks.)
- **Real-time Dashboard Visualization:** (Attack distribution charts/ Recent attack activity & Real-time toast alerts)
- **False Positive Detection Logic:** (Basic false-positive analysis was implemented to identify requests that may appear malicious but are actually legitimate user inputs.)

Conclusion:

The SentinelShield project successfully demonstrates the practical implementation of a lightweight Intrusion Detection System (IDS) and Web Application Firewall (WAF). The system is capable of inspecting incoming HTTP requests, detecting malicious payloads, monitoring abusive traffic behavior, generating logs, and providing real-time alerts through a dashboard interface.

The SentinelShield project successfully demonstrates the practical implementation of a lightweight Intrusion Detection System (IDS) and Web Application Firewall (WAF). The system is capable of inspecting incoming HTTP requests, detecting malicious payloads, monitoring abusive traffic behavior, generating logs, and providing real-time alerts through a dashboard interface.

--END--